

Translating code with Large Language Models and human-in-the-loop feedback

Gabriele Dario De Siano, Anna Rita Fasolino[✉], Giancarlo Sperli^{✉*}, Andrea Vignali[✉]

Department of Electrical Engineering and Information Technology (DIETI), University of Naples Federico II, Via Claudio 21, Naples, Italy

ARTICLE INFO

Keywords:

Large Language Model
Human-centered AI
Code translation
Software engineering

ABSTRACT

Context: In recent years, the code translation task has arisen as one of the major software issues in maintaining software quality during migration over complex infrastructure. This task involves human subjects with different background knowledge and could introduce errors due to the semantic gap between the programming languages and the complexity of the task. Generative Artificial Intelligence (AI) showed good capabilities in code generation, albeit this is highly dependent on the human factor.

Objective: This paper investigates, from the human perspective, the use of three Generative AI tools (ChatGPT, Google Bard, and GitHub Copilot) in the context of translation tasks from code written in query languages to code written in framework-specific code languages, specifically focused on SQL dialects and PySpark. This translation is especially crucial during the migration from centralized architectures to cloud-based architectures.

Methods: We evaluate the usefulness of these tools, the quality of the generated code, and their impact on performance. The models are tested with queries of various type in three different SQL dialects considering three usage scenarios of increasing complexity. It involves 15 participants with diverse programming backgrounds, who aim to solve tasks by interacting multiple times with the tools and manually changing the code.

Results: The findings show a positive performance, demonstrating their reliability in generating coherent translations, achieving 100% precision in most tasks with a slight decrease in more complex scenarios, and producing well-documented code, with a response time of under 2 min, with Google Bard responding 50% faster than the others.

Conclusion: In conclusion, this paper establishes a methodology and both quantitative and qualitative metrics for evaluating how generative AI tools streamline code translation, shifting the emphasis from production to refinement. It underscores the importance of continuously improving these tools to integrate them into developers' workflows and to provide guidelines for intelligent use.

1. Introduction

Generative AI tools are rapidly gaining traction in software engineering, offering the potential to significantly boost productivity [1]. However, challenges remain in ensuring consistent high-quality code output [2]. While these state-of-the-art models can automate software engineering productivity tasks like code translation, documentation generation, and method autocompletion, their indeterministic nature can sometimes lead to errors or incomplete code [3].

Large Language Models (LLMs) take advantage of patterns learned from vast amounts of information, helping to augment human performance and decision-making, entering the field of human-centered AI [4]. However, it is the user's responsibility to verify the correctness

of the answers that can be subjected to hallucinations or bias of the model [5]. Hence, the use of LLM tools in performing real tasks cannot be fully automatic but may require considerable human intervention. Human intervention is essential for: (a) engineering an appropriate prompt to formulate the request, (b) evaluating the response obtained and deciding whether it is adequate, or not, (c) establishing criteria for evaluating the adequacy of the response obtained, etc. The evaluation of outcome effectiveness may require demanding verification activities, such as in tasks where the LLM is used to generate software code that must meet specific quality criteria. The human operator will have to conduct additional static and dynamic code analysis activities and, possibly, also changes and corrections of the obtained result, before

* Corresponding author.

E-mail addresses: ga.desiano@studenti.unina.it (G.D. De Siano), fasolino@unina.it (A.R. Fasolino), giancarlo.sperli@unina.it (G. Sperli), andrea.vignali@unina.it (A. Vignali).

<https://doi.org/10.1016/j.infsof.2025.107785>

Received 12 September 2024; Received in revised form 28 April 2025; Accepted 12 May 2025

Available online 31 May 2025

0950-5849/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

reusing the obtained code. The type and iterations of human intervention required to obtain an acceptable LLM response can greatly affect the effectiveness and efficiency of the performed task, and thus the usability of the LLM.

Several works investigating the use of LLMs in software engineering tasks have been reported in the literature [6]. Although most of the works deal with software development (56.65%), they predominantly focus on code generation tasks rather than code translation tasks [6]. Specifically, among the 247 articles analyzed on software development, only 12 discuss software translation, while 118 are dedicated to code generation. In addition, many of the translation tasks described in the literature focus on the translation of code written in traditional programming languages (e.g., Java, C/C++, Go, and Python) into languages belonging to the same family [7,8]. However, there is a notable gap in research exploring the translation of code written in query languages (e.g., SQL) to framework-specific languages (e.g., PySpark) used for database development. Instead, researchers have concentrated predominantly on generating code (e.g., SQL) from natural language descriptions [9].

Rather than assessing the human element in the translation process, the literature focuses on optimizing the approach's effectiveness. Although a model may be accurate, it may not be able to process certain input types or in certain circumstances [6]. These metrics may not capture certain qualitative aspects of the model, such as its interpretability, robustness, or sensitivity to specific types of errors. The limits of some of the commonly used evaluation metrics in the field of LLM are further demonstrated by some of the most recent studies on LLMs for software engineering tasks [10], in which researchers modified some evaluation metrics to measure models' performance.

Evaluating the impact of these LLM techniques in code translation tasks executed in the context of migration processes is another relevant research challenge [11] not yet adequately addressed in the literature.

Human interaction evaluation (HIE) describes the effects of the interactions between humans and AI systems [12] and has a relevant impact on the software engineering landscape [13]. Hence, in our work, we define and implement an HIE method that focuses on the interaction mechanism between the person and the model, capturing both intermediate processes and final outputs. We investigate the usability of several LLM tools in a software code translation task by taking into account the human perspective. We focus on the specific context of translation tasks from query languages to framework-specific languages, which is relevant in migrations of systems from on-site premises to the cloud [11].

We performed a study that involved three popular LLM tools and human subjects who were assigned the task of translating query code into framework-specific languages utilizing the LLM tools. In the study, we investigated the iterative interaction process with the LLMs performed by the subjects until they completed the task under their judgment. We evaluate the usability of the tools according to the users' point of view, their effectiveness in producing quality code, and the overall efficiency of an LLM-supported code translation process.

Summarizing, the main contributions of this paper are:

- A method based both on qualitative and quantitative metrics to evaluate the usability of LLM tools in translating code from one programming language to another;
- A representation of the translation task that explicitly integrates human feedback to refine LLM outcomes until the end of the task;
- The evaluation of Generative AI tools' effectiveness and their capabilities in improving the quality of a developer's work through an experiment involving human subjects having different backgrounds and skills.

The remainder of this work is structured as follows. In Section 2 we introduce a background about LLMs, code generation and translation tasks. Section 3 provides an analysis of the state-of-the-art approaches

focusing on LLM tool evaluations, while Section 4 presents the experiment we performed and the methodology we followed to plan and execute it. Section 5 shows the results of our study. Lastly, in Section 6 we provide a summary of the main insights and key findings of our work.

2. Background

We provide the theoretical background by describing LLMs and code translation in Sections 2.1 and 2.2, respectively.

2.1. Large language models

LLMs are a type of AI model characterized by billions of parameters and trained on colossal amounts of text data to perform complex real-world tasks that go beyond traditional language modeling capabilities [14], also excelling in situations with limited or even zero annotations [15].

However, they can suffer from hallucinations, confidently presenting incorrect details as facts. Furthermore, training and running these models requires significant computational resources, which limits accessibility, making them expensive to deploy for real-world applications [16].

A key aspect in generative systems is *Prompt engineering* [17]. The *prompt* (input provided by the user to instruct the model to generate a response or output) plays a crucial role in the interaction with generative systems as it determines the direction and nature of the generated response to obtain useful and satisfying results.

The taxonomy of prompt engineering patterns reported in [18] offers a foundational framework to clarify the nature, roles, and implications of prompt engineering in software engineering tasks.

2.2. Code translation

Code translation refers to converting code from one programming language or format to another, often to facilitate interoperability between systems or to migrate legacy codebases to modern environments [19]. Code translation involves interpreting source code and generating equivalent code in a different language or dialect. In recent years approaches in unsupervised machine translation have been leveraged to train a fully unsupervised neural network [8,19,20]. In the following we will focus on the family of code translation approaches that exploit LLM tools, which we formalize in Definition 1.

Definition 1. Let P_{src} represent the prompt containing the source code in the original programming language. Let T_{LLM} be the function that translates P_{src} to P_{tgt} , the prompt in the target programming language. $T_{LLM}(P_{src}) \rightarrow P_{tgt}$. Where P_{tgt} represents the prompt formulated in the target programming language.

2.3. Code translation processes based on LLMs

Code translation using LLMs can be modeled as an iterative process including three main steps [19]:

- Prompt definition;
- LLM output generation;
- LLM response evaluation.

The human subject will be involved in the process both in the prompt definition step and in the LLM output evaluation one that aims to determine whether the code produced by the LLM presents the expected characteristics and whether the iterative process can be terminated, or not.

3. Related work

Recent studies related to code generation from natural language focus on analyzing the code generation capabilities assisted by LLMs and similar tools, including *ChatGPT*, *Google Bard*, *GitHub Copilot*, and *Amazon CodeWhisperer* [21,22]. The common goal is to evaluate the generated code's quality and efficiency through metrics such as validity, correctness, security, reliability, and maintainability, along with the ethical concerns of LLMs [23,24].

One key challenge in code generation using LLMs is establishing robust evaluation methods [25,26]. One approach is the automated evaluation using coding skill-improving sites or well-known datasets to assess the correctness of the code. For instance, GPT-3.5 and Bard were evaluated in generating Java code using a free site of live Java coding problems¹ where GPT-3.5 demonstrated superior performance by correctly generating code for about 90.6% of function descriptions compared to Bard's 53.1% [27]. Similarly, GitHub Copilot, Amazon CodeWhisperer, and ChatGPT have been evaluated using the HumanEval dataset centered on Python code [28]. In these evaluations, ChatGPT achieved the highest correctness in generated code (65.2%), followed by GitHub Copilot (46.3%) and Amazon CodeWhisperer (31.1%) [29]. In general, ChatGPT shows promise in code generation, especially for simpler programs, but struggles with more complex issues, while GitHub Copilot and Amazon CodeWhisperer have shown improvements but still have limitations. Another approach consists of the human-centered evaluation of the code generated using a sample of participants. An example of this, is a study that examined the generation of Python code by ChatGPT through two investigations using separate samples and participants: (i) a quantitative study focused on measurable aspects of code, while (ii) a qualitative study focused on user experiences and perceptions [30]. The results highlighted significant differences in the quality of code generated by AI compared to human solutions, with similarities in terms of accuracy and readability.

Other studies have focused on the application of generative AI models to the context of source code translation between different programming languages. One such study presented a fully unsupervised neural transcompiler trained on source code from GitHub projects and demonstrates high accuracy in translation between languages such as C++, Java, and Python [31]. Another study explored the application of generative techniques to source code generation, emphasizing the need for user interface features [32]. Similarly, further research investigated the impact of generative models on translating code from Java to Python. Despite imperfect model outputs, the presence of model assistance led to a reduction in errors in code produced by software engineers [33].

However, dealing with imperfect outputs is still an open challenge. Although prompts improve LLM effectiveness without modifying model parameters, it is challenging their design for downstream tasks [34].

To address this problem, the paradigm of *prompt-based learning* was introduced in natural language processing [35]. This approach is based on directly modeling the probability of text and offers model adaptation to new scenarios with little or no labeled data.

In Table 1, we summarize the above analyzed studies that focused on the evaluation of LLMs in code generation or translation tasks. The Table reports the experimental protocol, the main results achieved, and the list of pros and cons.

Many analyzed works focus on maximizing the effectiveness of the approach rather than evaluating the human contribution to the translation process. Even if Adamson and Bägerfeldt [30] shows interest in this topic, it puts more emphasis on the analysis of the differences between code generated by the experts and the generative AI. Similarly, the analysis made by Weisz et al. [33] is limited to evaluating the effectiveness of the translations, neglecting the broader aspects of user

engagement and iterative refinement of the output of LLM. This shows a gap in studies that investigate the impact of using these LLM tools in the translation process from the point of view of industrial experts. Moreover, many of the translation tasks described in the literature focus on the translation of code written in traditional programming languages (e.g., Java, C++, and Python) towards languages belonging to the same family. However, there is a lack of works that study the translation task from query languages (e.g., SQL) to framework-specific languages (e.g., PySpark) used for development. Instead, researchers have predominantly concentrated on generating code (e.g., SQL) from natural language descriptions, as illustrated by Li et al. [9]. Evaluating the impact of these LLM techniques in code generation tasks executed in the context of migration processes is another relevant research challenge [11] not yet adequately addressed in the literature.

4. Experimental methodology

The methodology was inspired by the methodological framework for conducting experiments in software engineering described by Wohlin et al. [36]. We started with the definition of the goal of the experiment (Section 4.1), to ensure that important aspects are defined before the planning and execution of the experiment take place. Hence, we proceeded with the experiment planning, which required the selection of the context of the experiment (Section 4.3), of dependent and independent variables (Section 4.4), selection of subjects (Section 4.5), definition of the experiment design (Section 4.6), and instrumentation (Section 4.7). We conducted the experiment and analyzed the obtained results.

4.1. Goal definition

The goal of our study is to investigate the use of LLM tools in the context of translation tasks from code written in query languages to code written in framework-specific code languages, exploiting human interaction. In this way, human subjects have to iteratively provide feedback to refine the LLM outcomes. We focus on the human perspective to investigate the *acceptability* of the LLM tools in the translation process and the impact of prompt engineering on the generated code quality.

4.2. Research questions

We refined the goal of our study by the following research questions:

RQ1. *How is the usability of LLM tools in the software translation process?* The objective of evaluating the usability of the tools focuses on a thorough examination of user interaction with each tool, with a particular emphasis on the ease of use of the LLM in the code translation process (*intuitiveness*). This includes not only the code translation phase but also the *ease* in detecting and correcting any *errors*.

RQ2. *How is the quality of the code generated by LLM tools?* With this question we want to evaluate the inner quality of the generated code using a prompt template. The code quality is examined by considering critical aspects that influence its reliability and maintainability, including *Code complexity*, *Overall code quality*,² *Code readability*, *Code optimization*.

RQ3. *How effective and efficient are LLM tools in the code translation task?* With this question, we want to investigate the effectiveness and efficiency of the tools in the context of automatic code translation, delving into the degree of success and accuracy. Effectiveness is evaluated by considering *translation accuracy* and *error handling*. Each of these

¹ <https://codingbat.com/java>.

² The overall code quality refers to the whole codebase health, including legacy code generated during the different interactions with the tool.

Table 1

Related work overview.

Paper	Task	Experimental protocol	Results	Pros and Cons
Destefanis et al. [27]	Java code generation from function descriptions	64 Java function descriptions were chosen from various sections of CodingBat.com. GPT-3.5 and Bard were prompted with each description to generate code, which was then assessed for accuracy	GPT-3.5 outperforms Bard in 4 out of 5 categories, providing correct translation 90.6% of the time, compared to Bard's 53.1%	- Decrease of performance and difficulty in error checking in dealing with complex tasks.
Adamson and Bäckerfeldt [30]	Evaluation of Python code generated by ChatGPT	Accuracy, quality, and readability comparison between code generated by ChatGPT and human-written code. Use of semi-structured interviews for a qualitative assessment of ChatGPT's usability	ChatGPT: Accuracy: 4.117 Quality: 3.266 Readability: 3.203 Human-written solution: Accuracy: 4.32 Quality: 3.57 Readability: 3.438	+Good performance in assigned tasks +Quick response -Difficulty in adhering to specific guidelines -Introduction of unnecessary changes -Decreased readability with the addition of new features -Need for continuous development
Yetiştiren et al. [29]	Generation and evaluation of Java and C++ code	Code generation capabilities of GitHub Copilot, Amazon CodeWhisperer, and ChatGPT were evaluated using the HumanEval benchmark	Code Correctness: <i>ChatGPT</i> : 65.2% <i>GitHub Copilot</i> : 46.3% <i>Amazon CodeWhisperer</i> : 31.1% Improvements in New Versions: <i>GitHub Copilot</i> : +18% <i>Amazon CodeWhisperer</i> : +7%	GitHub Copilot: +Good results in generating correct and partially correct solutions -9.1 min to eliminate code smells Amazon CodeWhisperer: +The fastest-to-fix code smells -Lowest success rate among evaluated tools ChatGPT: +Highest success rate among evaluated tools +Consistent performance with minimal functions and parameters -8.9 min to eliminate code smells -Decrease in performance when provided with only functions and parameters
Lachaux et al. [31]	Code translation between different programming languages	<i>TransCoder</i> is trained using a reinforcement learning method on monolingual source code from GitHub. The tested dataset includes 852 parallel functions with corresponding unit tests	Greedy Decoding - Accuracy: C++ → Java: 48.3% Java → Python: 50.5% Python → C++: 47.7% Beam Search Decoding - Accuracy: C++ → Java: 82% Java → Python: 65% Python → C++: 81.2%	+TransCoder significantly outperforms baselines in terms of computational accuracy +Beam Search improves accuracy +Simple and easily extendable approach -Expertise in source or target languages not required -Choice among hypotheses in the beam should be improved
Weisz et al. [33]	Evaluating the usefulness of generative AI in improving the quality of code translation	Quantitative and qualitative data are collected from 32 software engineers to assess the effectiveness of generative AI support for code translation from Java to Python.	The use of AI improved code translation performance, but subjective perception may not fully reflect the benefit.	+Positive rating of the quality of translations, despite the errors -Difficulty in error checking, and concerns about formatting and style of AI-generated code
Weisz et al. [32]	Translation of source code using generative techniques	Use of unsupervised neural machine translation (NMT) models in translating source code. Feedback from 11 software engineers was gathered through interviews.	Software engineers welcomed a generative model for code translation, even if it produces imperfect results. Interface tools were appreciated for explaining model decisions	-Decrease in the quality of code produced by an NMT model according to the existing best practice in software engineering.
Liu et al. [35]	Prompt-Based Learning	Analysis of various paradigms in the development of statistical natural language processing techniques, proposing unified mathematical notations, and organizing existing research on various aspects.	It is argued that prompt-based learning is a promising new paradigm that could represent another significant shift in how we perceive NLP.	- Issues in defining prompts to improve the effectiveness of the task.
Our study	Code translation from SQL dialects to PySpark	We investigate, from the human perspective, the use of ChatGPT, Google Bard, and GitHub Copilot varying LLM, Database, query and scenario.	LLMs achieve 100% precision in most scenarios, and produce well-documented code, with a response time of under 2 min, with Google Bard responding 50% faster than the others.	+We use quantitative and qualitative metrics for evaluation +We involve subjects with different background knowledge +100% precision in most scenarios +Response time under 2 min

plays a crucial role in determining the overall quality of the translation process. Efficiency, on the other hand, focuses on analyzing various

characteristics that influence their overall performance such as *time* taken to generate translations.

4.3. Experiment context

Based on the goal definition, we select the environment in which the experiment will be executed, i.e. the context of the experiment [36]. To define our context we select:

- The LLMs to be considered;
- The code translation problem to be executed;
- The LLM usage scenarios to be analyzed;
- The category of subjects to be involved in the study.

4.3.1. LLM selection

The LLMs chosen are **ChatGPT** by OpenAI,³ **Google Bard**,⁴ and **GitHub Copilot**.⁵ ChatGPT is selected for its versatility and strong contextual understanding, making it suitable for technical troubleshooting. Google Bard is incorporated due to its integration with real-time search capabilities and advanced natural language processing, providing up-to-date information retrieval. GitHub Copilot is included for its specialized code assistance within the development environment, facilitating efficient code generation and reducing the overhead associated with coding tasks. ChatGPT and Google Bard will be tested in their chatbot configurations to simulate their ease of interaction in conversational use cases. In contrast, GitHub Copilot will be utilized within the Visual Studio Code⁶ environment to replicate a realistic integrated software development scenario.

4.3.2. Code translation task

We consider translation tasks from code written in SQL query languages for relational databases to code written in framework-specific query languages. We focus on SQL dialects and PySpark, a Python API for Apache Spark⁷ that enables real-time and large-scale data processing in a distributed environment using Python. This translation is especially crucial during the migration from centralized architectures to cloud-based architectures, as it facilitates the transition.

We chose to vary the types of databases to minimize response bias from users regarding specific database types [37]. This approach led us to select 3 of the most widely used databases, ensuring a broader and more representative evaluation [38,39]. We selected **MySQL**, **PostgreSQL**, and **Oracle XE21c** databases. MySQL was chosen for its simplicity. PostgreSQL was selected for its advanced features, such as support for complex queries and full ACID⁸ compliance. Oracle XE21c was chosen for its powerful features and seamless integration with the Oracle ecosystem, offering a scalable solution for code migration.

On top of the databases, five different query types were designed to operate primarily on data types handled differently in the chosen SQL dialects. The queries can be categorized in terms of operation performed (**selection**, **aggregation**, **join**, **nested query**, and **view**).

4.3.3. LLM usage scenarios selection

To ensure a comprehensive evaluation of the three generative tools, we developed **Base**, **Intermediate**, and **Advanced** usage scenarios to simulate realistic interactions based on the strictness of the requirements that the generated code had to satisfy. The Base scenario focuses on generating a working code. The Intermediate scenario, instead, aims to produce not only working but also well-structured code (e.g., subdivision in functions and classes). The Advanced scenario aims to generate not only working and well-structured, but also secure (e.g., robust to SQL injection), readable (e.g., indented and properly commented), and maintainable (e.g., with a proper level of modularity) code.

4.3.4. Category of subjects

Despite static and automatic evaluations can prompt LLMs using human-written prompts and assess model outputs using annotations, interactive evaluations involve humans directly engaging with the model. In these evaluations, humans can be seen as subjects who prompt the model and actively respond to its outputs through multiple interactions [25].

In our experiment, we decided to consider subjects either from academia or industry, who could represent different categories of personnel who might find themselves faced with the task of generating SQL query code in industry. These subjects include not only more traditional software professionals with a specific background and expertise in SQL coding but also the so-called “citizen developers” who can exploit a no-code environment [40] to assemble custom enterprise Software-as-a-Service applications with little or no programming skills [41]. AI and Generative AI are indeed pushing low-code into the world of increasingly autonomous software creation.

4.4. Variable selection

The independent variables are those variables that we can control and change in the experiment. The independent variables in our study are the LLM and the LLM usage scenarios.

The effect of the treatments is measured in the dependent variables. The dependent variables we select include: LLM usability, code quality, effectiveness, and efficiency of the LLM tools. For evaluating the dependent variables we use the metrics defined in Section 4.7.3.

4.5. Selection of subjects

For defining our sample we used *Convenience sampling* [36]: the nearest and most convenient persons are selected as subjects.

We enrolled 15 participants both in academia and in industry, having different backgrounds and skills. They were recruited on a voluntary basis. Though of limited size, our sample can be considered valuable due to the well-known difficulties in recruiting more subjects having the requested characteristics [42] and the necessary willingness [43] in software engineering experiments.

In Table 2 we show a summary of information about participants. Specifically, 7 subjects were enrolled in a PhD program in Computer Engineering, and 2 were MSc students in Computer Engineering with database programming experience derived from their studies. Four more subjects were IT professionals with 1–5 years of experience in the industry, bringing a practical and consolidated perspective. We involved two participants coming from different branches with no technical profile within an IT company with no specific background in the field of Computer Science, allowing for coverage across a wide spectrum of skills. As to the demography, 8 individuals are between 25 and 29 years old, while the remaining 7 are between 30 and 34 years old.

Although we did not select the participants based on specific constraints, we studied the characteristics of our sample according to the suggestions provided Lenarduzzi et al. [42]. Before initiating the experiments, all participants were given a preliminary questionnaire that explored their knowledge about specific technologies (Fig. 1) and various programming languages (Fig. 2). In the questionnaire, they could respond with a value from 1 to 5, where 1 indicates the lack of knowledge and 5 indicates maximum confidence in their knowledge.

Among the aspects investigated in the questionnaire, participants were also asked to assess their previous knowledge of the LLM tools under study (Fig. 3), measuring their familiarity with these resources.

It is interesting to note that among the tools under study, ChatGPT is the most widely known to all participants. In contrast, none of the participants reported a major previous experience with GitHub Copilot. To ensure maximum confidentiality, all collected information was processed anonymously, thereby safeguarding the participants' privacy and maintaining the highest standards of data security.

³ Version 3.5.

⁴ Accessed on December 2023.

⁵ Accessed on December 2023.

⁶ <https://code.visualstudio.com>.

⁷ <https://spark.apache.org>.

⁸ ACID is the acronym of Atomicity, Consistency, Isolation, and Durability.

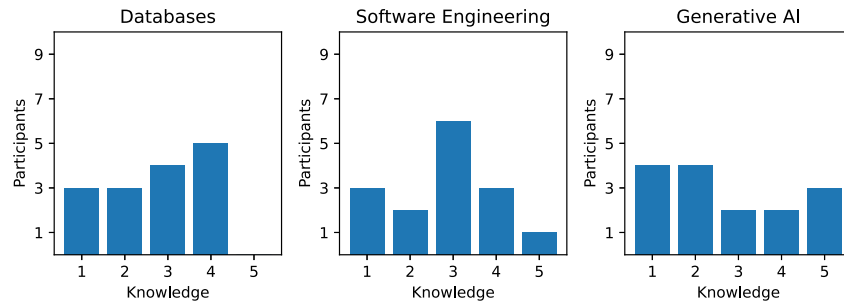


Fig. 1. Participants' knowledge of software technologies measured on a scale from 1 (lack of knowledge) to 5 (Maximum confidence in their knowledge).

Table 2

Summary of participants' information. The Background column indicates the participants' educational or professional background in Computer Engineering: PhD, MSc, Industry (professional experience in industry), and None (no relevant academic or industry background in Computer Engineering).

Participant	Age (years)	Background
P1	26	MSc
P2	30	MSc
P3	31	Industry
P4	27	PhD
P5	27	PhD
P6	33	PhD
P7	31	PhD
P8	25	PhD
P9	28	PhD
P10	25	PhD
P11	30	None
P12	30	Industry
P13	29	Industry
P14	29	Industry
P15	31	None

by the Research Questions reported in 4.2. Since we selected three different LLMs (ChatGPT, Google Bard, and GitHub Copilot) and three different usage scenarios (Simple, Intermediate, Advanced) in which we intended to execute the translation tasks, we had nine treatments, each one consisting of one possible value of the pair (LLM, Usage Scenario). We ensured that each pair (LLM, Usage Scenario) was used in the execution of at least 5 different tasks. To distribute the tasks among the subjects, we let each subject free to choose from 2 to 5 of the 15 possible Code translation tasks reported in Section 4.3.2, consisting of the pair (Database, Query Type). We removed the assigned tasks from the list as they were chosen by the subjects to avoid the same task repetitions. We adopted this arbitrary task distribution strategy, rather than a randomized one, to ensure the necessary engagement of the subjects in the assigned tasks. Allowing participants to choose tasks they find interesting or comfortable increases motivation and reduces dropout rates, which is critical in studies requiring active participation. It reduced frustration and bias from mismatched expertise since participants might have struggled with randomly assigned tasks that were too difficult or irrelevant to their skills. This strategy was closer to real-world scenarios where developers often have some autonomy in tool selection and task difficulty.

4.7. Instrumentation

The instruments for an experiment are of three types: objects, guidelines, and measurement instruments. The former consists of the artifacts involved in the experiment whilst the latter is necessary to guide the experimenters through the procedure, whose outcomes are collected through the measurement instruments.

4.7.1. Objects

The first type of objects involved in the experiment were three different implementations of a database based on three specific SQL dialects (*MySQL*, *PostgreSQL*, *Oracle XE21c*) that threat differently the data types.

The database was designed based on an industrial context, focusing on an electronic equipment manufacturing company. The database structure and description can be found at the following link.⁹ The implemented structure considers the differences in data types supported by different databases, ensuring that each of them faithfully reflects the complexity of the considered industrial context. The entities were carefully designed to reflect the complexity and diversity of data at the expense of simpler relationships. This careful design allows for coherent data structuring, considering variations in SQL dialects, while facilitating navigation and extraction of interconnected information through the defined relationships. The databases were then populated with randomly generated elements using the Faker module.¹⁰

Fig. 2. Participants' knowledge of programming languages measured on a scale from 1 (lack of knowledge) to 5 (Maximum confidence in their knowledge).

4.6. Experiment design

As design type, we chose the *one factor with more than two treatments*, where the factor consists of the acceptability of the LLM in the considered translation task, according to the goal stated in 4.1 and specified

⁹ <https://osf.io/he3xq/files/osfstorage/6800fe2b54bb0d1fccc4400>.

¹⁰ <https://faker.readthedocs.io/en/master/>.

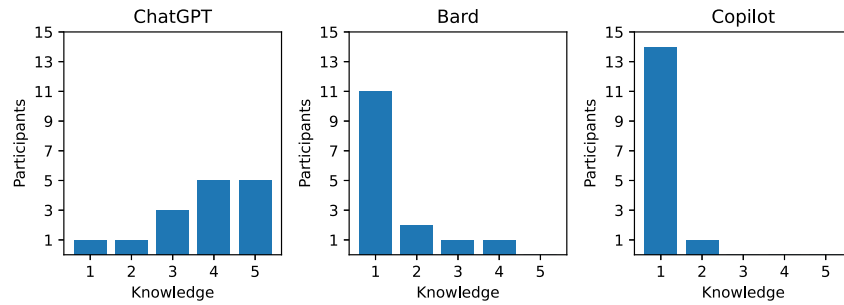


Fig. 3. Participants' knowledge of LLM tools measured on a scale from 1 (lack of knowledge) to 5 (Maximum confidence in their knowledge).

Request Name	Aggregation Query translation from MySQL to Copilot.
Problem Description	I have to translate an aggregation query from SQL language to Python.
Query to Translate	<pre>SELECT ae.TipoProdotto YEAR(ae.DataProduzione) AS AnnoProduzione, SUM(LENGTH(ae.ManualeUtente)) AS DimensioneTotaleManuali FROM ApparecchiatureElettroniche ae GROUP BY ae.TipoProdotto, YEAR(ae.DataProduzione)</pre>
Additional Details	Database used is MySQL. I want to split up the resulting Python code in functions. I want to handle exceptions. I want to include comments for each resulting section of the code.
Technical Requirements	Use the library mysql-connector and divide the code in different modules.
Security Level	Do not include sensitive data within the code and avoid SQL-injection attacks.
Extra	The output must be a json file. Organize every module in directories, adding also one directory for testing. Generate tests for the security assessment.
Code Structure	Code structure is the following: <pre>#import of the useful libraries #global variable definition #definition of the function that connects to the database #definition of the functions for the execution of the program #main</pre>

Fig. 4. Prompt example. The prompt is organized in a template where the first column guides the user during the compilation.

We considered 3 database-specific implementations of the 5 different types of queries introduced in Section 4.3.2. They are reported at the following link⁹.

LLMs' strong sensitivity to changes in prompt wording is a commonly mentioned barrier to their use as human substitutes in subjective tasks [44]. Using prompt patterns is a well-known practice in interacting with LLMs as reported in several recent papers [35,45]. Prompt patterns allow a more effective, controllable, and repeatable interaction with the models while leaving freedom to the human subject on how to fill it out. Hence, we defined a template reporting the different parts of the prompt to be compiled in each different scenario, shown in Fig. 4.

The main part of the template comprises a short and unique identifier (**Request Name**), a description of the problem (**Problem Description**), the SQL query to translate (**Query to Translate**). The template includes any useful information to understand the context of the request, such as the type of database, exception handling, or the level of detail in comments (**Additional Details**), the use of certain libraries or frameworks, or adopting specific design patterns (**Technical Requirements**), or the security level for the generated code (**Security Level**). The final part of the template covers any additional options, such as a brief description of expected behavior, an example of generated output, or a request to generate verification tests (**Extra**), and the **Code Structure**, relevant only to GitHub Copilot, where providing a basic code structure to guide the generation process. Each field is designed to facilitate both the generation of requests by participants following the three scenarios, and the comprehension of the utilized tools.

4.7.2. Experimental procedure

Our experimental procedure is composed of 4 stages, mainly relying on the *Prompting Generation*, *Tools Interaction*, *Output Verification*, and *Metrics Calculation*.

Prompting generation. To ensure a precise interaction with the tools, in the Prompt generation stage the participant had to use the template reported in Fig. 4.

Tools interaction. The tools interaction phase focused on submitting the requests using the template to each LLM and evaluating the LLM's response generation time. Each request had to be executed in separate conversation sessions for the ChatGPT and Google Bard tools, as well as a new session was opened in a Jupyter Notebook for GitHub Copilot to avoid context persistence and ensure that the tools were not influenced by previous requests. After obtaining the responses, each subject had to perform a detailed response analysis to verify the correct understanding of the context by the LLM tool and identify any gaps in the generated code before moving to the next testing phase.

Output verification. The code generated by the tools was compiled and executed and the participant's task was to identify and resolve any compilation error and then execute the obtained code until he was satisfied with the exhibited behavior. Overcoming errors were handled flexibly, allowing participants to decide whether to address the problem through a new interaction with the tools or resolve it independently. Once the code had been generated, the subjects executed their code under the guidance of a tutor, who assisted them in verifying the correctness of the output. The termination criteria of the output verification phase were determined by the user. The experiment ended either when the user considered the task resolved (the code runs

without compilation and execution errors) or when the user decided to stop further prompting. To accomplish the Output verification phase of the procedure performed by the two subjects not having Computer Science experience, the intervention of one of the authors was needed to support them.

Metrics calculation. Once the correct operation of a translation was confirmed, the code was archived in a GitHub repository. Subsequently, we measured the efficiency of the tools and the quality of the generated code with specific evaluation tools (**SonarQube**, **Radon**). For the evaluation of the tools' usability, a satisfaction questionnaire was developed, consisting of nine questions aimed at exploring various tool features from the perspective of usability and the overall user experience. The questions were designed to investigate specific aspects related to the interaction with generative AI tools, allowing for an in-depth assessment of usability. An overview of the questionnaire is shown in [Appendix](#). Each participant, at the end of the experiments, assessed the tools by answering these questions, assigning a numerical rating based on the following 5 levels:

1. *Strongly disagree*
2. *Slightly disagree*
3. *Neither agree nor disagree*
4. *Slightly agree*
5. *Strongly agree*

Once the participants were satisfied with the obtained code at the end of the *Output Testing* step, the *Metric Calculation* step was executed by one of the authors. Hence, we did not impose a maximum time limit for task execution because our focus is on analyzing the number of code changes and the code generation times of the LLMs. We are not interested in evaluating the total time taken by the subject to complete the task. The task completion time is heavily influenced by several factors, including the responses provided by the LLM, the user's background, and how the prompt template was completed.

Before running the experiments, we explained the phases of the procedure to the participants, demonstrating how to interact with the three tools and how to distinguish between the three scenarios we designed. We showed them the prompt template and provided detailed instructions on correctly filling it out. We explained in detail the usage scenarios and how to correctly complete tasks for each of them (see *Output Testing* paragraph for more details). In [Fig. 5](#) we show the activity diagram of the experimental procedure from the point of view of the participant.

4.7.3. Metric framework

As regards RQ1, we used as a metric the **Satisfaction value**, extracted from a questionnaire composed of 9 questions ([Appendix](#)). The questions were designed to investigate specific aspects related to the interaction with generative AI tools, allowing for an in-depth assessment of usability.

Regarding RQ2, we decided to evaluate the quality of translated code using the metrics reported in [Table 3](#). Most of them present integer or float values, while Cyclomatic Complexity, Maintainability, Reliability, and Security are measured in non-uniform score ranges identified by letters, where A is considered the best score according to the documentation of Radon and SonarQube.

To measure them, we used Radon¹¹ and SonarQube.¹² Radon is a Python library that provides tools for analyzing the quality of source code, also computing cyclomatic complexity, and the number of source lines of code (SLOC), and provides assessments of code maintainability. SonarQube can assess key aspects such as reliability, security, maintainability, and test coverage.

As regards RQ3, we used the following metrics:

- **Response time** of the tool in generating the answer measured in seconds;
- Number of **Changes** to the translated code to achieve a correct and functional translation. It is defined as the sum of the number of interactions (**INTs**) with the tool required to get to a solution considered acceptable by the user and the number of times the user changed the code without the assistance of the tool (**CWTs**);
- Number of **Errors** encountered during code compilation;
- **Precision** in generating code translation on the first user interaction. This value is expressed as a percentage and is defined in Eq. (1)

$$Precision = \left(1 - \frac{0.7 \cdot CWTs + 1.2 \cdot INTs + 1.5 \cdot Errors}{SLOC} \right) \cdot 100 \quad (1)$$

5. Results

We discuss the experimental results according to the research questions defined in [Section 4.2](#) on the basis of the qualitative and quantitative metrics (see [Section 4.7.3](#)).

5.1. RQ1: How is the usability of LLM tools in the software translation process?

After completing the assigned tests, participants contributed to the study by responding to the usability questionnaire for generative AI tools presented in the previous section. The participants' responses were synthesized through [Tables 4, 5, 6](#), each corresponding to one of the tools used, providing a detailed insight into the user experience with each tool.

The answers range between 1 (Strongly Disagree) and 5 (Strongly Agree), N/A indicates that the participant either did not get assigned a task for a specific LLM or did not need the LLM for additional support (Q5) or correcting issues or errors in the generated code (Q7).

In [Table 4](#), ChatGPT emerged that 100% of participants positively evaluated the accessibility of the service (Q1) and the intuitiveness of the tool (Q2), emphasizing its ease of use. 80% expressed satisfaction with the speed at which ChatGPT generated responses (Q3), while all users found the level of comprehension of the generated code satisfactory (Q4). 90% appreciated comments and recommendations in the code (Q5), deeming them helpful.

As to Google Bard (see [Table 5](#)), 80% of the participants positively evaluated the accessibility of the service (Q1) and the intuitiveness (Q2). 80% of users were satisfied with the response speed (Q3), and the same percentage found the level of comprehension of the generated code satisfactory (Q4). 70% of users found comments and recommendations in the code helpful (Q5).

For GitHub Copilot (see [Table 6](#)), 70% of the participants positively evaluated the intuitiveness of the tool (Q2) and appreciated comments and recommendations in the code (Q5). 80% expressed satisfaction with the speed at which Copilot generated responses (Q3). However, only 60% found the level of comprehension of the generated code satisfactory (Q4), and the accessibility to the tool was criticized (Q1). It is worth noting that P11 and P15, who lacked programming experience, did not complete the specific questionnaire for GitHub Copilot. This decision was made to avoid having them interact with a tool that requires specialized knowledge.

In [Fig. 6](#), we compare the qualitative results of the questionnaire for each question. We report mean and median values to provide a more comprehensive understanding of the data distribution, particularly to highlight any skewness within the vote distribution within the boxplots.

ChatGPT and Google Bard achieved perfect scores in terms of usability (Q2) due to their intuitive chatbot interfaces, while Copilot received a median score of 2. Notably, Copilot was the least familiar tool among participants, whereas ChatGPT was the most recognized, however, despite being less well-known within the sample, Bard performed on par with ChatGPT because it employs a similar conversational interface. In

¹¹ <https://radon.readthedocs.io/en/latest/index.html>.

¹² <https://docs.sonarsource.com/sonarqube/latest/>.

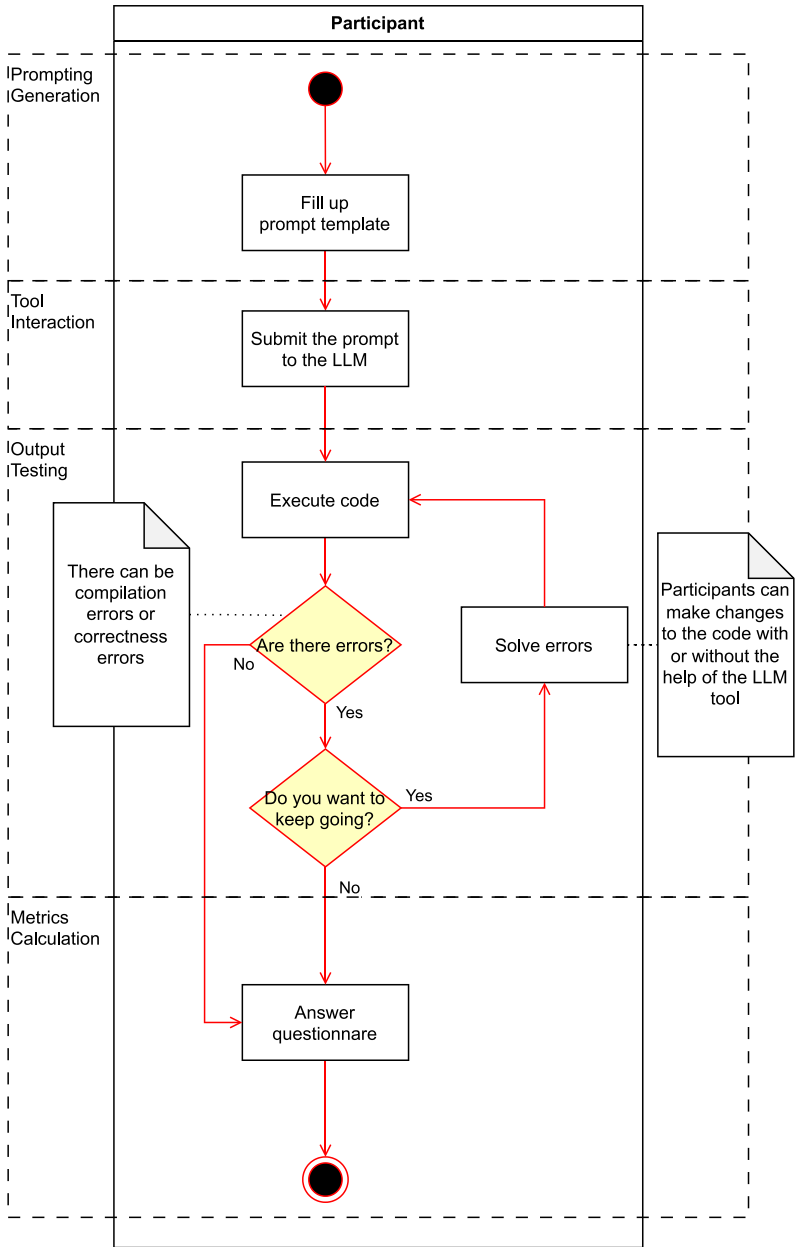


Fig. 5. Activity diagram of the experimental procedure from the point of view of the participant.

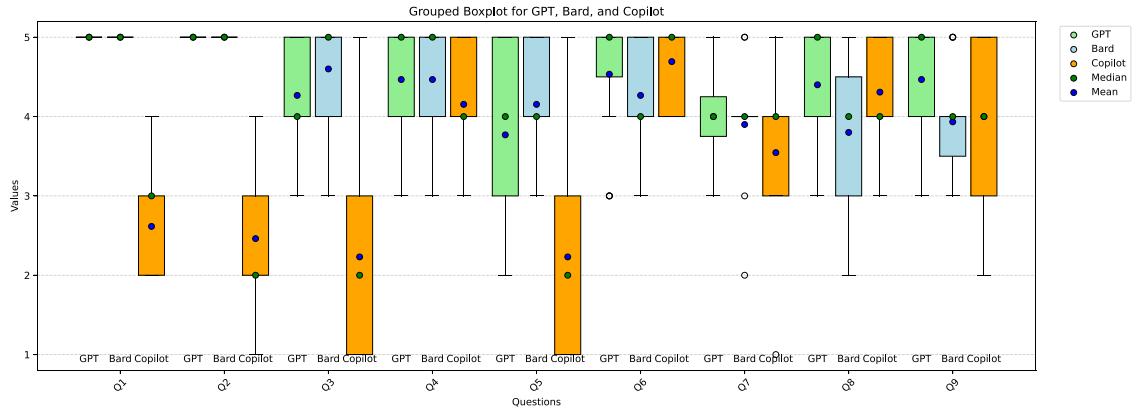


Fig. 6. Box plots of the average answers to the questionnaire per question and tool.

Table 3

RQ2 metrics table. This table is divided into three sections: (i) Size metrics, (ii) Complexity metrics, and (iii) Other metrics. In the first two sections, are listed symbols representing the number of operators (η_1 for distinct, N_1 for total), and operands (η_2 for distinct, N_2 for total). The sum of the total operators and operands represents the program length N .

Attribute	Metric	Definition	Scale	Tool
Size	Source Lines of Code (SLOC)	Number of lines of source code	Integer	raw (Radon)
	Comment Density (CD)	Percentage of comments in the code compared to the total number of lines	Float	raw (Radon)
Size	Halstead Volume (V)	$V = N \log_2 \eta$	Float	hal (Radon)
Complexity	Difficulty (D)	$D = \frac{\eta_1}{2} * \frac{N_2}{\eta_2}$	Float	hal (Radon)
Complexity	Cyclomatic Complexity (CC)	Number of decisions a block of code contains + 1	A – F	cc (Radon)
Maintainability	Maintainability Index (MI) Score [46]	$MI = MAX(0, (171 - 5.2 \cdot \ln(V) - 0.23 \cdot (CC) - 16.2 \cdot \ln(SLOC) + \sin(\sqrt{2.4 \cdot CD})) \cdot 100/171)$	A – C	mi (Radon)
Reliability	Bugs	Number of bugs in the code	A – E	SonarQube
Security	Security	Assessment of the code vulnerabilities	A – E	SonarQube
Efficiency	Average Code Execution Time	Average execution time of the code	Float	%%timeit (Jupyter Notebooks)

Table 4

Satisfaction questionnaire results: ChatGPT.

Participant	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9
P1	5	5	5	5	5	5	5	5	5
P2	5	5	5	5	5	5	4	4	5
P3	5	5	4	4	4	3	3	3	3
P4	5	5	4	4	2	3	4	3	3
P5	5	5	5	5	4	5	N/A	5	5
P6	5	5	3	4	3	5	N/A	5	4
P7	5	5	4	4	3	4	N/A	4	4
P8	5	5	4	4	4	5	5	5	5
P9	5	5	4	4	5	5	4	4	5
P10	5	5	4	5	5	5	N/A	5	5
P11	5	5	4	5	N/A	5	3	5	4
P12	5	5	5	5	3	5	N/A	5	5
P13	5	5	4	5	3	5	4	5	5
P14	5	5	5	5	3	5	N/A	5	5
P15	5	5	4	3	N/A	3	N/A	3	4

Table 6

Satisfaction questionnaire results: GitHub Copilot.

Participant	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9
P1	3	3	3	5	5	5	5	5	3
P2	3	2	5	4	4	5	4	4	3
P3	3	3	2	4	2	4	4	4	4
P4	2	2	1	4	1	4	4	4	3
P5	4	4	3	3	2	5	1	5	4
P6	2	2	1	4	2	5	N/A	5	2
P7	2	2	2	5	2	4	N/A	4	5
P8	3	3	2	5	2	5	4	4	3
P9	2	2	4	4	3	5	5	4	5
P10	3	4	3	5	3	5	3	5	5
P11	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
P12	3	2	1	4	1	5	3	3	5
P13	2	1	1	3	1	4	3	4	5
P14	2	2	1	4	1	5	3	5	5
P15	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Table 5

Satisfaction questionnaire results: Google Bard.

Participant	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9
P1	5	5	4	5	5	5	5	5	4
P2	5	5	5	4	4	5	4	4	4
P3	5	5	3	4	3	3	2	3	3
P4	5	5	5	5	4	3	4	4	3
P5	5	5	5	5	4	5	N/A	5	5
P6	5	5	4	4	4	5	4	5	5
P7	5	5	5	5	3	4	N/A	4	4
P8	5	5	5	5	5	4	4	4	4
P9	5	5	4	4	4	5	4	3	5
P10	5	5	5	3	4	4	3	3	3
P11	5	5	5	5	N/A	5	N/A	5	4
P12	5	5	5	5	4	4	5	3	3
P13	5	5	5	5	5	4	N/A	4	4
P14	5	5	5	5	5	4	4	2	4
P15	5	5	4	3	N/A	4	N/A	3	4

general, all three tools received positive ratings for overall satisfaction (Q9) with a preference towards ChatGPT. Copilot excels in context understanding compared to the others (Q6) while the trend inverts for problem-solving assistance (Q7). For the adaptability to different scenarios, Bard performs moderately, while GPT and Copilot have an average score greater than 4 (Q8). We can conclude that the usability of the three tools is influenced only slightly by users' familiarity with the tools themselves, and more significantly by their interfaces, which prioritize human-like interaction and feedback over code generation from a structure. ChatGPT stands out due to its high ratings for accessibility and intuitiveness. It also emerges that Google Bard should work on improving code comments and recommendations and GitHub Copilot needs to improve accessibility and code comprehension.

5.2. RQ2: How is the quality of the code generated by LLM tools?

We analyze the results related to the code quality based on the qualitative and quantitative metrics defined in Table 3 computed on each experiment performed in the Base 5.2.1, Intermediate 5.2.2, and Advanced 5.2.3 scenarios. In Table 7 we show the number of tasks performed for each scenario, tool, and database. Since we did not control the assignment of tasks to the subjects, the data in the table also shows the subjects' preferences in choosing the tasks.

It is worth noting that human subjects preferred to complete tasks related to the Base scenario (25 tasks), while Intermediate and Advanced have been selected 20 and 19 times respectively. In the Base scenario, there is a preference for ChatGPT, while in the Intermediate scenario, GitHub Copilot is favored. In the Advanced scenario, no clear preference for either tool emerges when selecting tasks. From the database point of view, the most used ones are MySQL and PostgreSQL with 24 and 28 tasks respectively, while OracleXE21c tasks have been selected only 12 times, showing a preference towards open source databases.

5.2.1. Base scenario

Table 8 reports the data obtained for the basic scenario, where columns refer to the metrics described in Section 4.7.3.

If we consider the translations of the same type of queries (e.g., Aggregation), we see that ChatGPT exhibits **higher** values of SLOC score while Google Bard and GitHub Copilot produce more compact code. Although **Cyclomatic Complexity** could not be evaluated in all cases due to the simplicity of the generated code, the few instances where it was assessed showed that all the tools produced code with low complexity (A rank). Due to the simplicity of the code, also the **Volume** and **Difficulty** metrics turned out to be zero most of the time, as the tool used to calculate them could not assess them. The results of **Reliability**

Table 7
Statistics about task accomplished by scenario, tool, and database.

Scenario	Tool	Database	Number of task	Total
Base	ChatGPT	MySQL	4	10
		PostgreSQL	5	
		OracleXE21c	1	
	Google Bard	MySQL	2	7
		PostgreSQL	4	
		OracleXE21c	1	
	Github Copilot	MySQL	3	8
		PostgreSQL	4	
		OracleXE21c	1	
Intermediate	ChatGPT	MySQL	2	6
		PostgreSQL	2	
		OracleXE21c	2	
	Google Bard	MySQL	2	5
		PostgreSQL	2	
		OracleXE21c	1	
	Github Copilot	MySQL	2	9
		PostgreSQL	5	
		OracleXE21c	2	
Advanced	ChatGPT	MySQL	3	6
		PostgreSQL	2	
		OracleXE21c	1	
	Google Bard	MySQL	3	7
		PostgreSQL	3	
		OracleXE21c	1	
	Github Copilot	MySQL	3	6
		PostgreSQL	1	
		OracleXE21c	2	

Table 8

Base scenario: Quality of generated code.

LLM	Database	Query	SLOC	CC	V	D	MI	Reliability	Security	CD	Av. Exe. T.
ChatGPT	MySQL	Select	21	A	0	0	A	C	A	20	12.2 ± 1.58
	MySQL	Aggregation	59	A	4.75	0.5	A	C	A	12	
	MySQL	Join	71	A	0	0	A	C	A	9	21.6 ± 1.75
	MySQL	Nested	33	A	0	0	A	A	A	15	9.6 ± 1.51
	PostgreSQL	Select	22		0	0	A	C	A	21	77.4 ± 6.82
	PostgreSQL	Aggregation	25		0	0	A	C	A	16	63.2 ± 4.40
	PostgreSQL	Join	115	A	38.04	2	A	C	E	4	
	PostgreSQL	Nested	37		0	0	A	C	A	13	103 ± 4.96
	PostgreSQL	Vista	69		0	0	A	C	A	14	101 ± 10.20
	Oracle XE21c	Select	12		0	0	A	C	A	21	115 ± 4.98
Google Bard	MySQL	Aggregation	14	A	0	0	A	C	A	25	15.6 ± 1.18
	MySQL	Vista	30		4.75	0.5	A	C	A	18	21.2 ± 2.71
	PostgreSQL	Select	13		0	0	A	C	A	23	103 ± 16.80
	PostgreSQL	Aggregation	23		0	0	A	C	A	21	100 ± 5.28
	PostgreSQL	Nested	30		0	0	A	C	A	0	104 ± 9.15
	PostgreSQL	Vista	31		0	0	A	C	A	12	120 ± 30.30
	Oracle XE21c	Vista	20		0	0	A	C	A	14	
GitHub Copilot	MySQL	Aggregation	11		0	0	A	C	A	43	10.4 ± 1.36
	MySQL	Join	117		0	0	A	C	A	34	11.8 ± 1.11
	MySQL	Nested	61		0	0	A	C	A	0.07	20.8 ± 1.76
	PostgreSQL	Select	15		0	0	A	C	A	41	110 ± 21.10
	PostgreSQL	Aggregation	18		0	0	A	C	A	41	81.7 ± 13.30
	PostgreSQL	Join	64		0	0	A	C	A	47	124 ± 20.90
	PostgreSQL	Nested	40		0	0	A	C	A	38	94.5 ± 11.10
	Oracle XE21c	Select	12		0	0	A	C	A	46	53.8 ± 6.44

show that the code may contain at least one major bug (C rank), in contrast with the results regarding **Security** and **Maintainability**, where all tools meet the established criteria with positive evaluations (A rank). For **Comment Density**, all tools produce well-commented code, with GitHub Copilot standing out for significantly higher density, indicating more integrated documentation in the code. Observing the **Average Code Execution Time**, no significant differences are noted between the tools. GitHub Copilot shows a slight optimization of the code compared to the other tools, manifesting itself with slightly shorter execution times for the same queries. On the contrary, Google Bard tends to

have longer execution times, suggesting a different approach to code optimization.

5.2.2. Intermediate scenario

Table 9 reports the data obtained for the basic scenario, where columns refer to the metrics described in Section 4.7.3.

The SLOC metric follows the trend of the base scenario, where ChatGPT presents higher values, while both Google Bard and GitHub Copilot generate more concise code, suggesting a preference for a more concise approach once again. The **Cyclomatic Complexity** value is maintained at acceptable levels (A rank). Only in one case was higher

Table 9

Intermediate scenario: Quality of generated code.

Tool	Database	Query	SLOC	CC	V	D	MI	Reliability	Security	CD	Av. Exe. T.
ChatGPT	MySQL	Join	75	A	0	0	A	C	A	9	13.70 $\pm$ 2.80
	MySQL	Nested	50	A	4.75	0.5	A	C	A	11	8.98 $\pm$ 1.77
	PostgreSQL	Select	49	A	24.00	1.5	A	C	A	15	86.00 $\pm$ 5.02
	PostgreSQL	Nested	62	A	4.75	0.5	A	C	A	0.16	54.80 $\pm$ 8.95
	Oracle XE21c	Select	48	B	2.00	0.5	A	C	A	16	159.00 $\pm$ 48.20
Google Bard	Oracle XE21c	Aggregation	54	A	13.93	0.5	A	C	A	1	83.60 $\pm$ 16.30
	MySQL	Nested	41	A	0	0	A	C	A	31	8.10 $\pm$ 1.18
	MySQL	Vista	28		0	0	A	C	A	11	
	PostgreSQL	Select	28	A	4.75	0.5	A	C	A	29	81.40 $\pm$ 24.40
	PostgreSQL	Nested	64	A	4.75	0.5	A	C	A	27	72.70 $\pm$ 4.88
GitHub Copilot	Oracle XE21c	Aggregation	30	A	4.75	0.5	A	C	A	38	32.80 $\pm$ 2.66
	MySQL	Aggregation	38	A	4.75	0.5	A	C	A	31	24.20 $\pm$ 12.20
	MySQL	Nested	29	A	23.26	0.6	A	C	A	15	9.51 $\pm$ 0.58
	PostgreSQL	Select	37	A	0	0	A	C	A	32	98.00 $\pm$ 17.80
	PostgreSQL	Aggregation	40	A	0	0	A	C	A	35	63.70 $\pm$ 8.92
	PostgreSQL	Join	65		0	0	A	C	A	45	93.00 $\pm$ 9.58
	PostgreSQL	Nested	48	A	0	0	A	C	A	35	62.30 $\pm$ 11.20
	PostgreSQL	Vista	71	A	4.75	0.5	A	C	A	39	
	Oracle XE21c	Select	36	A	15.50	1	A	C	A	3	155.00 $\pm$ 18.60
	Oracle XE21c	Aggregation	32	A	0	0	A	C	A	33	46.00 $\pm$ 17.80.00

Table 10

Advanced scenario: Quality of generated code.

Tool	Database	Query	SLOC	CC	V	D	MI	Reliability	Security	CD	Av. Exe. T.
ChatGPT	MySQL	Select	54	A	4.75	0.5	A	A	A	0.01	
	MySQL	Aggregation	58	A	4.75	0.5	A	C	A	17	4.87 $\pm$ 0.31
	MySQL	Join	129	A	38.03	2	A	C	E	8	47.90 $\pm$ 5.91
	PostgreSQL	Aggregation	49	A	0	0	A	A	E	4.7	
	PostgreSQL	Nested	74	A	15.51	1	A	C	A	6	50.10 $\pm$ 7.66
	Oracle XE21c	Vista	51	A	4.75	0.5	A	A	A	12	
Google Bard	MySQL	Select	32	A	4.75	0.5	A	C	A	0	4.96 $\pm$ 0.58
	MySQL	Aggregation	48	A	4.75	0.5	A	C	A	12	4.09 $\pm$ 0.19
	MySQL	Join	95	A	4.75	0.5	A	C	A	0	10.40 $\pm$ 2.64
	PostgreSQL	Select	22	A	4.75	0.5	A	C	A	9	86.80 $\pm$ 3.84
	PostgreSQL	Join	85	A	15.51	1	A	C	A	9	
	PostgreSQL	Nested	108	A	36.00	1.33	A	C	A	10	41.80 $\pm$ 4.45
	Oracle XE21c	Select	63	A	15.51	1	A	C	A	20	45.20 $\pm$ 10.10
GitHub Copilot	MySQL	Select	95	A	17.81	1.18	A	C	A	14	4.95 $\pm$ 0.31
	MySQL	Aggregation	58	A	4.75	0.5	A	C	A	31	9.14 $\pm$ 0.64
	MySQL	Join	90	A	4.75	0.5	A	C	A	38	4.55 $\pm$ 0.16
	PostgreSQL	Nested	108	A	36.66	2	A	C	A		40.80 $\pm$ 8.22
	Oracle XE21c	Select	48	A	4.75	0.5	A	C	A	17	37.50 $\pm$ 9.72
	Oracle XE21c	Aggregation	29	A	0	0	A	C	A	3	79.10 $\pm$ 37.7

Cyclomatic Complexity observed (B rank), attributed to a more complex request to the tool. The **Volume** metric suggests that Google Bard tends to generate shorter code snippets, reflecting greater conciseness. In contrast, ChatGPT and Github Copilot show more extensive volume values indicating more verbose code generation. Observing the **Difficulty**, all three tools continue to generate code with low values, indicating good code comprehensibility. **Maintainability**, **Reliability** and **Security** do not vary from the base scenario. Regarding **Comment Density**, Google Bard and Github Copilot produce well-commented code, unlike ChatGPT, which struggles to comment on the code at the expense of a more detailed explanation in response generation. The **Average Code Execution Time** does not present huge differences between the tools. Contrary to the base scenario, Github Copilot does not show significant code optimization compared to the other two tools, while Google Bard exhibits better execution times. This could be attributed to the fact that the prompts contain more detailed requests.

5.2.3. Advanced scenario

Table 10 reports the data obtained for the basic scenario, where columns refer to the metrics described in Section 4.7.3.

Regarding the **SLOC** metric, ChatGPT and Github Copilot present considerable values, while Google Bard, for the majority of requests, generates more compact code, definitely confirming a preference for

a more concise approach. The **Cyclomatic Complexity** maintains acceptable levels because even when more articulated requests are made, the generated code does not exhibit structures that further complicate it. For the **Volume**, ChatGPT continues to show a tendency to generate shorter code snippets, reflecting greater conciseness. Github Copilot, on the other hand, exhibits more extensive volume values, indicating more verbose code generation. The **Difficulty** of the generated code remains relatively low for all three tools, suggesting good comprehensibility. ChatGPT and Github Copilot stand out for an even lower overall difficulty, indicating greater intuitiveness of the produced code. The metrics of **Maintainability**, **Security** and **Reliability** report similar results as described for the previous scenarios, except in some ChatGPT cases where the reliability reaches the A rank and the security falls on the E rank. Regarding **Comment Density**, Github Copilot maintains a significantly higher density compared to the other tools. In contrast, ChatGPT and Google Bard, by presenting an external graphical interface to the development IDE, comment and explain the code outside the generated snippet. Considering the **Average Code Execution Time**, the more the requests to Google Bard are detailed, the more it can generate optimized code, almost reaching the results obtained by Github Copilot. However, ChatGPT also improves its code optimization when subjected to more detailed requests.

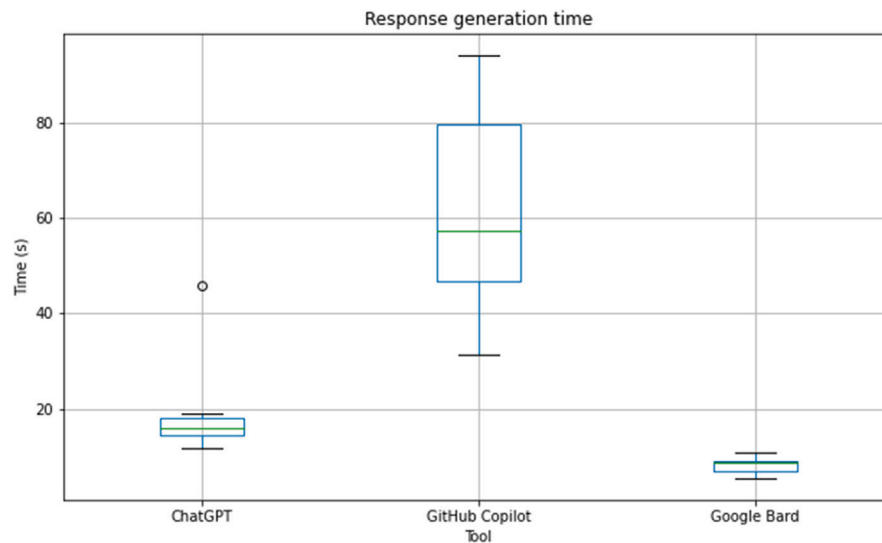


Fig. 7. Base scenario: Response generation time.

5.3. RQ3: How effective and efficient are LLM tools in the code translation task?

In this paragraph, we analyze the results based on the quantitative metrics defined for the RQ3 of each experiment performed on the three scenarios. The statistics of the tasks are the same as RQ2 (see Table 7). In terms of **response time** of the *base scenario* in Fig. 7, ChatGPT shows an acceptable performance, with a range from 11 s to 46 s. Google Bard stands out for shorter times, with the majority of translations completed in less than 10 s. However, GitHub Copilot records longer times, with some values approaching 100 s, indicating a potential performance discrepancy among queries.

In Table 11 we show the resulting **changes**, **errors**, and **precision** of the base scenario. ChatGPT demonstrates high precision in the majority of cases, with some exceptions in queries on the Oracle XE21c database, similar to GitHub Copilot. Google Bard exhibits remarkable efficiency, achieving 100% precision in various query categories and databases. Analyzing the number of changes and errors, both ChatGPT and Google Bard show a positive trend with a reduced need for corrections. GitHub Copilot, while generally efficient, exhibits some variations in the number of changes and errors, but overall maintains an acceptable level of performance.

For the *intermediate scenario*, Fig. 8 further highlights the differences in the **response time** between the three tools, showing the superiority of Google Bard in the speed of generation of a response.

Table 12 shows how tools in the intermediate usage scenario positively handle more complex requests while maintaining good precision and requiring a limited number of modifications. ChatGPT demonstrates a good ability to handle moderately complex requests, achieving 100% precision in various query categories across different databases. Even in the presence of some necessary modifications, the quantity remains limited, and the overall efficiency is high. Google Bard shows good adaptability, maintaining a high level of precision. The required modifications are limited, and the maintainability of the generated code is positively evaluated. GitHub Copilot, while achieving positive results, exhibits greater variability in performance. In some queries on PostgreSQL, for example, a significant number of modifications and slightly lower precision are observed. However, the ability to handle this type of request is evident in other cases.

Fig. 9 shows the **response time** of the advanced scenario. It is worth noticing that also in this scenario Github Copilot performs the worst, while Google Bard takes a few seconds to generate a response.

Table 13 shows that ChatGPT exhibits notable precision in addressing complex requests. While some specific situations require adjustments, the performance remains generally high, showcasing its adaptability to advanced scenarios. Google Bard continues to demonstrate good efficiency. Even in more advanced situations, it maintains a high degree of precision, albeit with a moderate number of required changes. GitHub Copilot also performs well in the advanced scenario, sustaining high precision in translating queries. Despite the need for modifications in some instances, the overall efficiency is positive, reflecting its ability to manage complex requests effectively.

5.4. Threats to validity

Threats to internal validity concern issues that may indicate a causal relationship, although there is none. To mitigate the *Instrumentation* threat we carefully designed the materials and procedures used in the experiment and trained the subjects before they took part in the experiment. For mitigating the *selection* threat, we selected subjects who are representative of typical personnel employed in database translation and migration tasks, including both post-graduate and professionals with different backgrounds in database development, software engineering, and generative AI. Since the translation tasks were arbitrarily and not randomly assigned to subjects, we cannot exclude that the obtained results were biased by their background. To mitigate this threat we are aware that further studies will be necessary, where we may adopt randomized task assignments to avoid clustering around easier or harder tasks, require each participant to use multiple tools to reduce self-selection bias, and do pre-experiment training, to standardize initial exposure to the tools, so that differences in prior familiarity do not affect the results.

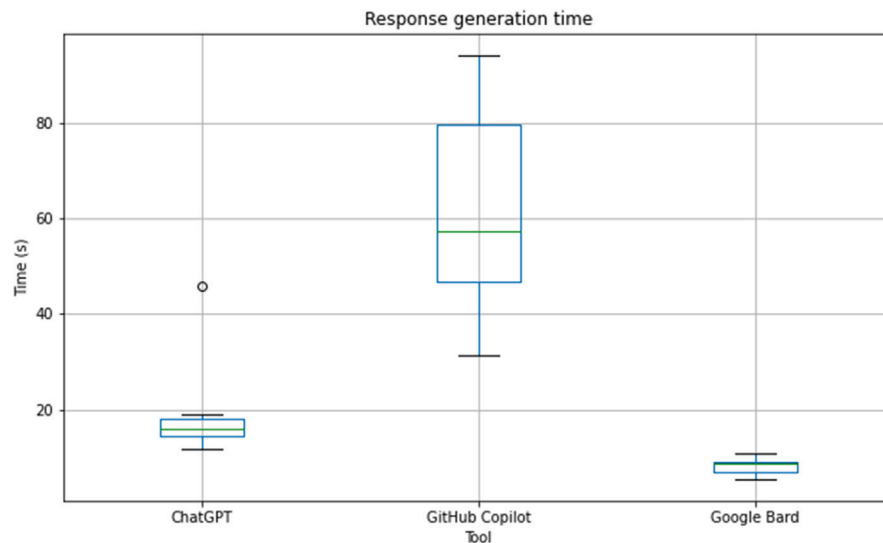
Threats to construct validity refer to the extent to which the operational measures that are studied represent what the researcher has in mind and what is investigated. To mitigate the mono-operational bias due to using a single operational variant during the experiment, we considered three popular LLMs and three RDBMS, five types of SQL queries having different degrees of complexity, and three types of scenarios for the LLM-human interaction tasks. To avoid the *mono-method* bias, we used multiple metrics offered by Radon and SonarQube to evaluate the quality of the generated code, as well as both subjective (questionnaires) and objective measures (e.g., number of compilation errors, number of correction changes) to evaluate the usability and effectiveness of the LLMs.

Threats to external validity concern the ability to generalize experimental results outside the experiment setting. The small size of the

Table 11

Base scenario: Performance metrics for effectiveness of tools.

Tool	Database	Query	Changes	Errors	Precision
ChatGPT	MySQL	Select	0	0	100.00%
	MySQL	Aggregation	3	0	94.00%
	MySQL	Join	0	0	100.00%
	MySQL	Nested	0	0	100.00%
	PostgreSQL	Select	0	0	100%
	PostgreSQL	Aggregation	0	0	100%
	PostgreSQL	Join	2	2	95.30%
	PostgreSQL	Nested	0	0	100.00%
	PostgreSQL	Vista	0	0	100.00%
Google Bard	Oracle XE21c	Select	5	2	25.00%
	MySQL	Aggregation	0	0	100.00%
	MySQL	Vista	0	0	100.00%
	PostgreSQL	Select	0	0	100.00%
	PostgreSQL	Aggregation	1	1	87.72%
	PostgreSQL	Nested	0	0	100.00%
	PostgreSQL	Vista	0	0	100.00%
	Oracle XE21c	Vista	0	0	100.00%
GitHub Copilot	MySQL	Aggregation	0	0	100.00%
	MySQL	Join	0	0	100.00%
	MySQL	Nested	3	3	90.00%
	PostgreSQL	Select	1	0	95.30%
	PostgreSQL	Aggregation	0	0	100.00%
	PostgreSQL	Join	0	0	100.00%
	PostgreSQL	Nested	0	0	100.00%
	Oracle XE21c	Select	1	0	94.17%

**Fig. 8.** Intermediate scenario: Response generation time.

participant sample threatens the generalizability of the study results. Another threat is that using only 5 types of SQL queries may not capture the complexity and variety of queries encountered in the real world. Considering only three types of LLM-human interaction scenarios, three LLMs and three RDBMS may be oversimplified, with respect to the variety of solutions available in the practice. To mitigate these threats, in future work, we intend to replicate our study involving more participants with diverse backgrounds, experiences, and contexts to improve representativeness. We aim to consider more interaction scenarios, LLMs, and DBMSs. We intend to repeat the study over time to assess how the results evolve with new versions of LLMs. We plan to replicate the experiment in real-world environments and integrate the results with data collected in practical work environments to increase their relevance.

LLMs are subject to various biases, including the Beauty Bias [47], which reflects a preference for visually appealing code. This phenomenon highlights how the stylistic quality of the generated code

can vary significantly across different models, suggesting the potential influence of aesthetic considerations on output preferences.

6. Discussion and conclusion

Current approaches for assessing LLMs' impact on translation tasks [30,33] focus on evaluating translation quality but overlook human-AI interaction. Moreover, they work on well-known procedural languages without investigating the capability of LLMs in supporting migrations to the cloud.

This paper investigated the use of Generative AI tools, specifically ChatGPT, Google Bard, and GitHub Copilot, for the automatic translation of SQL queries into PySpark language by defining and implementing an HIE method focused on the interaction mechanism between the person and the model. We evaluated the usability, effectiveness, and efficiency of three different LLM tools in the execution of the translation tasks, by considering the human perspective and feedback.

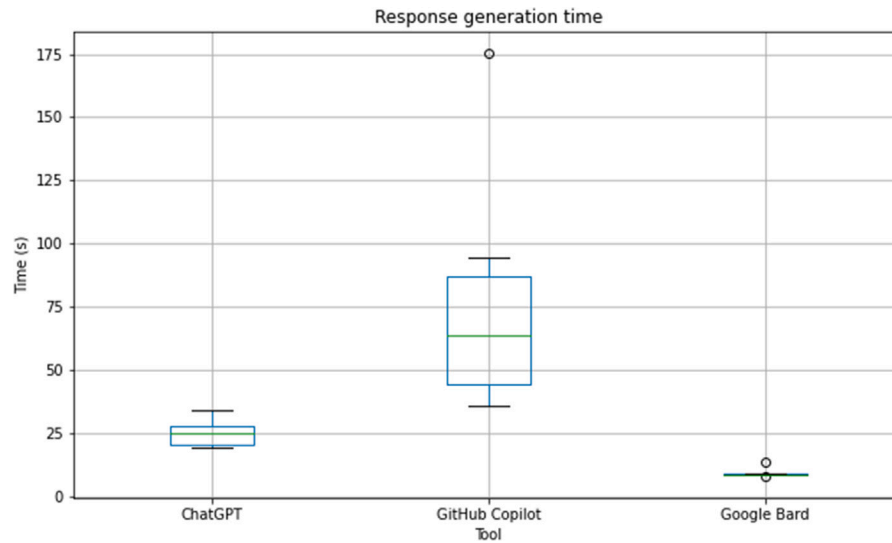


Fig. 9. Advanced scenario: Response generation time.

Table 12

Intermediate scenario: Performance metrics for effectiveness of tools.

Tool	Database	Query	Changes	Errors	Precision
ChatGPT	MySQL	Join	0	0	100.00%
	MySQL	Nested	0	0	100.00%
	PostgreSQL	Select	2	2	90%
	PostgreSQL	Nested	0	0	100.00%
	Oracle XE21c	Select	0	0	100.00%
	Oracle XE21c	Aggregation	0	0	100.00%
Google Bard	MySQL	Nested	2	1	91.70%
	MySQL	Vista	1	1	90.40%
	PostgreSQL	Select	2	2	84.00%
	PostgreSQL	Nested	0	0	100.00%
	Oracle XE21c	Aggregation	1	1	92.67%
GitHub Copilot	MySQL	Aggregation	0	0	100.00%
	MySQL	Nested	1	1	92.40%
	PostgreSQL	Select	3	3	81.0%
	PostgreSQL	Aggregation	0	0	100.00%
	PostgreSQL	Join	0	0	100.00%
	PostgreSQL	Nested	7	1	86.60%
	PostgreSQL	Vista	0	0	100.00%
	Oracle XE21c	Select	2	2	86.00%
	Oracle XE21c	Aggregation	0	0	100.00%

Table 13

Advanced scenario: Performance metrics for effectiveness of tools.

Tool	Database	Query	Changes	Errors	Precision
ChatGPT	MySQL	Select	0	0	100.00%
	MySQL	Aggregation	1	1	95.35%
	MySQL	Join	0	0	100.00%
	PostgreSQL	Aggregation	0	0	100.00%
	PostgreSQL	Nested	1	0	99.05%
	Oracle XE21c	Vista	6	6	69.30%
Google Bard	MySQL	Select	1	1	91.60%
	MySQL	Aggregation	4	4	79.60%
	MySQL	Join	2	2	94.00%
	PostgreSQL	Select	0	0	100.00%
	PostgreSQL	Join	3	2	92.80%
	PostgreSQL	Nested	4	1	93.50%
	Oracle XE21c	Select	5	2	85.71%
GitHub Copilot	MySQL	Select	2	1	96.40%
	MySQL	Aggregation	1	1	96.20%
	MySQL	Join	3	3	91.00%
	PostgreSQL	Nested	4	1	93.50%
	Oracle XE21c	Select	7	7	65.80%
	Oracle XE21c	Aggregation	1	0	95.90%

Table 14

Summary of research questions, findings, and their practical/research implications. Each row outlines one research question, its associated findings, and what these imply for practice and further investigation.

RQ#	Findings	Implications
RQ1	LLMs that offered an intuitive interface were preferred over less user-friendly LLMs	Usability is the key attribute for a subject to choose the most effective LLM for their specific task
RQ2	ChatGPT showed to be the best LLM to write reliable code, there is no clear winner in terms of execution time of code. Copilot showed how to manage the complexity of code written	Future development workflows might favor a combination of tools to exploit their strengths
RQ3	Bard showed to be the fastest, and ChatGPT the one that has the highest precision	ChatGPT and Bard are suitable for fast prototyping code

The research involved 15 participants aged between 25 and 35, all with diverse backgrounds in computer science. The analysis was conducted through three distinct scenarios, each with specific query requirements and complexities. The experimental results showed that three scenarios of increasing complexity (Base, Intermediate, Advanced) have very different results in terms of the quality of the generated code, performance of the involved LLM tools, and tool usability from the human experts' perspective.

In the base scenario, ChatGPT generated code with an average execution time 18% faster than Bard's, while remaining comparable to Copilot's. In the intermediate scenario, however, Bard outperformed both ChatGPT and Copilot, achieving execution times about 29% faster. In the advanced scenario, Copilot generated code that was 14% faster than the one generated by ChatGPT and 9% faster than the one generated by Bard. For the number of SLOC, the intermediate scenario produced between 34% and 40% more lines of source code than the base one, while for the advanced scenario, ChatGPT kept the same length of the code, while Bard and Copilot increased the amount of SLOC by about 60%. Overall, Bard excelled in speed and accuracy, ChatGPT demonstrated adaptability and reliability, and Copilot effectively managed complexity despite its variability and slower performance. When considering usability, ChatGPT and Bard stand out as the most accessible and intuitive tools, thanks to their highly user-friendly chatbot interface and strong performance across all usability metrics. In contrast, GitHub Copilot, while excelling in response speed and contextual understanding, falls short in accessibility and code comprehension, likely due to its design, which requires more advanced programming skills. A key takeaway is that conversational interfaces, like those in ChatGPT and Bard, improve usability by giving human-like interaction and feedback precedence over the creation of structured code. Finally, with respect to the translation task (RQ3), the precision of the translations revealed that, in both the base and advanced scenarios, the LLMs achieved translation accuracies exceeding 90% for MySQL and PostgreSQL, whereas performance remained suboptimal for Oracle XE21c.

Summarizing, ChatGPT and Google Bard's ability to provide textual explanations helped participants understand both the generated code and the general task context. Even when code compilation errors occurred, these tools offered adequate support by explaining the reasons for these errors. Bard demonstrated the fastest response times and highest accuracy, excelling in speed and precision. ChatGPT showed adaptability and reliability but required occasional adjustments, while Copilot managed complex queries effectively despite slower performance and greater variability. We summarize our findings and the relative implications in Table 14.

Our study addressed existing gaps in the literature by making two key contributions. First, we focused on the LLM-supported translation from query languages (e.g., SQL) to framework-specific languages

(e.g., PySpark) used in distributed systems, whilst state-of-the-art methods primarily treat the translation of well-known procedural programming languages [31]. Second, we conducted a human-centered analysis to explore how subjects' interactions with generative tools can facilitate task resolution. Unlike prior studies that mainly compare the effectiveness of LLMs [29–31], our approach involves user interactions and usability, providing deeper insights into human-centered AI.

Future research should focus on integrating advanced optimization techniques to produce relevant code to address a specific task. Establishing clear guidelines and best practices will ensure the secure and effective use of these tools, fostering collaboration between developers and AI. By refining Generative AI and promoting its responsible use, the software industry can unlock new frontiers of innovation.

CRediT authorship contribution statement

Gabriele Dario De Siano: Writing – original draft, Validation, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Anna Rita Fasolino:** Writing – original draft, Validation, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Giancarlo Sperli:** Writing – original draft, Validation, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Andrea Vignali:** Writing – original draft, Validation, Supervision, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been partially funded by the project NextGenerationEU via PNRR - DM 352 (CUP: E66G22000400009).

Appendix. Questionnaire

How much do you agree with the following sentences (1-(strongly disagree) and 5-(strongly agree))?

1. **Service Accessibility:** the access to the service was intuitive.
2. **Tool Usability:** the interaction with the tool was intuitive
3. **Response latency:** the latency of the interaction process required by the tool for generating a suitable response was acceptable

4. **Response Comprehensibility:** the level of understanding of the response generated by the tool was satisfactory
5. **Additional Support:** the additional support provided by the tool to understand and use the provided code, in terms of generated code comments, usage examples, and recommendations was helpful
6. **Context Understanding:** the tool achieves a good understanding of the specific context
7. **Problem-Solving Assistance:** the support offered by the tool in resolving any issues or errors in the generated code was helpful
8. **Adaptability to Different Scenarios:** the tool was able to generate appropriate code based on different usage scenarios
9. **Overall Satisfaction:** the user experience with the tool was satisfactory.

Data availability

The authors do not have permission to share data.

References

- [1] A. Moradi Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M.C. Desmarais, Z.M.J. Jiang, GitHub Copilot AI pair programmer: Asset or Liability? *J. Syst. Softw.* 203 (2023) 111734, <http://dx.doi.org/10.1016/j.jss.2023.111734>.
- [2] Z.M. Zain, S. Sakri, N.H.A. Ismail, Application of Deep Learning in Software Defect Prediction: Systematic Literature Review and Meta-analysis, *Inf. Softw. Technol.* 158 (2023) 107175, <http://dx.doi.org/10.1016/j.infsof.2023.107175>.
- [3] R. Choudhuri, D. Liu, I. Steinmacher, M. Gerosa, A. Sarma, How Far Are We? The Triumphs and Trials of Generative AI in Learning Software Engineering, in: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, Association for Computing Machinery, New York, NY, USA, 2024, <http://dx.doi.org/10.1145/3597503.3639201>.
- [4] D. Russo, S. Baltes, N. van Berkel, P. Avgeriou, F. Calefato, B. Cabrero-Daniel, G. Catolino, J. Cito, N. Ernst, T. Fritz, H. Hata, R. Holmes, M. Izadi, F. Khomh, M.B.K. rgaard, G. Liebel, A.L. Lafuente, S. Lambiasi, W. Maalej, G. Murphy, N.B. Moe, G. O'Brien, E. Paja, M. Pezzè, J.S. Persson, R. Prikladnicki, P. Ralph, M. Robillard, T.R. Silva, K.-J. Stol, M.-A. Storey, V. Stray, P. Tell, C. Treude, B. Vasilescu, Generative AI in Software Engineering Must Be Human-Centered: The Copenhagen Manifesto, *J. Syst. Softw.* 216 (2024) 112115, <http://dx.doi.org/10.1016/j.jss.2024.112115>.
- [5] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y.J. Bang, A. Madotto, P. Fung, Survey of Hallucination in Natural Language Generation, *ACM Comput. Surv.* 55 (12) (2023) <http://dx.doi.org/10.1145/3571730>.
- [6] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, Large language models for software engineering: A systematic literature review, *ACM Trans. Softw. Eng. Methodol.* (2023).
- [7] J. Pan, A. Sadé, J. Kim, E. Soriano, G. Sole, S. Flamant, SteloCoder: a decoder-only LLM for multi-language to python code translation, 2023, arXiv preprint [arXiv:2310.15539](https://arxiv.org/abs/2310.15539).
- [8] R. Pan, A.R. Ibrahimzade, R. Krishna, D. Sankar, L.P. Wassi, M. Merler, B. Sobolev, R. Pavuluri, S. Sinha, R. Jabbarvand, Lost in translation: A study of bugs introduced by large language models while translating code, in: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [9] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, M. Chenhao, G. Li, K. Chang, F. Huang, R. Cheng, Y. Li, Can LLM already serve as a database interface? A Big bench for large-scale database grounded text-to-SQLs, in: A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, S. Levine (Eds.), *Advances in Neural Information Processing Systems*, Curran Associates, Inc., pp. 42330–42357.
- [10] A. Singla, Evaluating ChatGPT and GPT-4 for visual programming, in: *Proceedings of the 2023 ACM Conference on International Computing Education Research-Volume 2*, 2023, pp. 14–15.
- [11] R. Zmigrod, S. Alamir, X. Liu, Translating between SQL Dialects for Cloud Migration, in: *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, in: ICSE-SEIP '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 189–191, <http://dx.doi.org/10.1145/3639477.3639727>.
- [12] L. Weidinger, M. Rauh, N. Marchal, A. Manzini, L.A. Hendricks, J. Mateos-Garcia, S. Bergman, J. Kay, C. Griffin, B. Bariach, et al., Sociotechnical safety evaluation of generative ai systems, 2023, arXiv preprint [arXiv:2310.11986](https://arxiv.org/abs/2310.11986).
- [13] D. Lo, Trustworthy and synergistic artificial intelligence for software engineering: Vision and roadmaps, in: *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, IEEE, 2023, pp. 69–85.
- [14] W.X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., A survey of large language models, 2023, arXiv preprint [arXiv:2303.18223](https://arxiv.org/abs/2303.18223).
- [15] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, S. Zhong, B. Yin, X. Hu, Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond, *ACM Trans. Knowl. Discov. Data* 18 (6) (2024) <http://dx.doi.org/10.1145/3649506>.
- [16] M.C. Rillig, M. Ågerstrand, M. Bi, K.A. Gould, U. Sauerland, Risks and benefits of large language models for the environment, *Environ. Sci. Technol.* 57 (9) (2023) 3464–3466.
- [17] C. Pornprasit, C. Tantithamthavorn, Fine-tuning and prompt engineering for large language models-based code review automation, *Inf. Softw. Technol.* 175 (2024) 107523, <http://dx.doi.org/10.1016/j.infsof.2024.107523>.
- [18] Y. Sasaki, H. Washizaki, J. Li, D. Sander, N. Yoshioka, Y. Fukazawa, Systematic literature review of prompt engineering patterns in software engineering, in: *2024 IEEE 48th Annual Computers, Software, and Applications Conference, COMPSAC*, 2024, pp. 670–675, <http://dx.doi.org/10.1109/COMPSAC61105.2024.00096>.
- [19] Z. Yang, F. Liu, Z. Yu, J.W. Keung, J. Li, S. Liu, Y. Hong, X. Ma, Z. Jin, G. Li, Exploring and unleashing the power of large language models in automated code translation, *Proc. ACM Softw. Eng.* 1 (FSE) (2024) <http://dx.doi.org/10.1145/3660778>.
- [20] B. Roziere, M.-A. Lachaux, L. Chatusot, G. Lample, Unsupervised Translation of Programming Languages, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems*, Vol. 33, Curran Associates, Inc., 2020, pp. 20601–20611.
- [21] X. Jiang, Y. Dong, L. Wang, F. Zheng, Q. Shang, G. Li, Z. Jin, W. Jiao, Self-planning code generation with large language models, *ACM Trans. Softw. Eng. Methodol.* (2024) <http://dx.doi.org/10.1145/3672456>.
- [22] X. Zhang, Y. Zhou, G. Yang, T. Han, T. Chen, Context-aware code generation with synchronous bidirectional decoder, *J. Syst. Softw.* 214 (2024) 112066, <http://dx.doi.org/10.1016/j.jss.2024.112066>.
- [23] J. Liu, C.S. Xia, Y. Wang, L. ZHANG, Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation, in: A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, S. Levine (Eds.), *Advances in Neural Information Processing Systems*, Vol. 36, Curran Associates, Inc., 2023, pp. 21558–21572.
- [24] M. Evtikhiev, E. Bogomolov, Y. Sokolov, T. Bryksin, Out of the BLEU: How should we assess quality of the Code Generation models? *J. Syst. Softw.* 203 (2023) 111741, <http://dx.doi.org/10.1016/j.jss.2023.111741>.
- [25] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, W. Ye, Y. Zhang, Y. Chang, P.S. Yu, Q. Yang, X. Xie, A Survey on Evaluation of Large Language Models, *ACM Trans. Intell. Syst. Technol.* 15 (3) (2024) <http://dx.doi.org/10.1145/3641289>.
- [26] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, Y. Lou, Evaluating Large Language Models in Class-Level Code Generation, in: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, Association for Computing Machinery, New York, NY, USA, 2024, <http://dx.doi.org/10.1145/3597503.3639219>.
- [27] G. Destefanis, S. Bartolucci, M. Ortu, A Preliminary Analysis on the Code Generation Capabilities of GPT-3.5 and Bard AI Models for Java Functions, 2023, arXiv preprint [arXiv:2305.09402](https://arxiv.org/abs/2305.09402).
- [28] M. Chen, J. Tworek, H. Jun, Q. Yuan, H.P.D.O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, 2021, arXiv preprint [arXiv:2107.03374](https://arxiv.org/abs/2107.03374).
- [29] B. Yetişiren, I. Özsoy, M. Ayerdem, E. Tüzün, Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT, 2023, arXiv preprint [arXiv:2304.10778](https://arxiv.org/abs/2304.10778).
- [30] V. Adamson, J. Bäckerfeldt, Assessing the effectiveness of ChatGPT in generating Python code, 2023.
- [31] M.-A. Lachaux, B. Roziere, L. Chatusot, G. Lample, Unsupervised translation of programming languages, 2020, arXiv preprint [arXiv:2006.03511](https://arxiv.org/abs/2006.03511).
- [32] J.D. Weisz, M. Muller, S. Houde, J. Richards, S.I. Ross, F. Martinez, M. Agarwal, K. Talamadupula, Perfection Not Required? Human-AI Partnerships in Code Translation, in: *Proceedings of the 26th International Conference on Intelligent User Interfaces*, IUI '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 402–412, <http://dx.doi.org/10.1145/3397481.3450656>.
- [33] J.D. Weisz, M. Muller, S.I. Ross, F. Martinez, S. Houde, M. Agarwal, K. Talamadupula, J.T. Richards, Better Together? An Evaluation of AI-Supported Code Translation, in: *Proceedings of the 27th International Conference on Intelligent User Interfaces*, IUI '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 369–391, <http://dx.doi.org/10.1145/3490099.3511157>.
- [34] P. Sahoo, A.K. Singh, S. Saha, V. Jain, S. Mondal, A. Chadha, A systematic survey of prompt engineering in large language models: Techniques and applications, 2024, arXiv preprint [arXiv:2402.07927](https://arxiv.org/abs/2402.07927).

- [35] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, G. Neubig, Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing, *ACM Comput. Surv.* 55 (9) (2023) <http://dx.doi.org/10.1145/3560815>.
- [36] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, A. Wesslén, *Experimentation in Software Engineering*, Springer, 2012, <http://dx.doi.org/10.1007/978-3-642-29044-2>.
- [37] D.E. O'Leary, Do large language models bias human evaluations? *IEEE Intell. Syst.* 39 (4) (2024) 83–87, <http://dx.doi.org/10.1109/MIS.2024.3415208>.
- [38] J. Wagner, A. Rasin, Forget about it: Batched database sanitization, in: *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing, SAC '24*, Association for Computing Machinery, New York, NY, USA, 2024, pp. 1441–1452, <http://dx.doi.org/10.1145/3605098.3636054>.
- [39] T. Taipalus, H. Grahni, H. Ghanbari, Error messages in relational database management systems: A comparison of effectiveness, usefulness, and user confidence, *J. Syst. Softw.* 181 (2021) 111034, <http://dx.doi.org/10.1016/j.jss.2021.111034>.
- [40] G. Juhás, L. Molnár, A. Juhásová, M. Ondříšová, M. Mladoniczky, T. Kováčik, Low-code platforms and languages: the future of software development, in: *2022 20th International Conference on Emerging ELearning Technologies and Applications, ICETA, 2022*, pp. 286–293, <http://dx.doi.org/10.1109/ICETA57911.2022.9974697>.
- [41] G.F. Hurlburt, Low-code, no-code, what's under the hood? *IT Prof.* 23 (6) (2021) 4–7, <http://dx.doi.org/10.1109/MITP.2021.3123415>.
- [42] V. Lenarduzzi, O. Dieste, D. Fucci, S. Vegas, Towards a methodology for participant selection in software engineering experiments: A vision of the future, in: *Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM, Association for Computing Machinery, New York, NY, USA, 2021*, <http://dx.doi.org/10.1145/3475716.3484273>.
- [43] A. Rainer, C. Wohlin, Recruiting credible participants for field studies in software engineering research, *Inf. Softw. Technol.* 151 (2022) 107002, <http://dx.doi.org/10.1016/j.infsof.2022.107002>.
- [44] L. Tjuatja, V. Chen, T. Wu, A. Talwalkwar, G. Neubig, Do LLMs exhibit human-like response biases? A case study in survey design, *Trans. Assoc. Comput. Linguist.* 12 (2024) 1011–1026, http://dx.doi.org/10.1162/tacl_a_00685.
- [45] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, Large language models for software engineering: A systematic literature review, *ACM Trans. Softw. Eng. Methodol.* 33 (8) (2024) <http://dx.doi.org/10.1145/3695988>.
- [46] D. Coleman, D. Ash, B. Lowther, P. Oman, Using metrics to evaluate software system maintainability, *Computer* 27 (8) (1994) 44–49.
- [47] G.H. Chen, S. Chen, Z. Liu, F. Jiang, B. Wang, Humans or LLMs as the Judge? A Study on Judgement Biases, 2024, [arXiv:2402.10669](https://arxiv.org/abs/2402.10669).